Mathematics of Machine Learning

Over-Fitting





Overfitting, regularisation, feature selection

Outline

1. **Over-fitting?**
2. Controlling Complexity
3. Hidden structure
4. Regularisation
5. Double Descent



Over-fitting

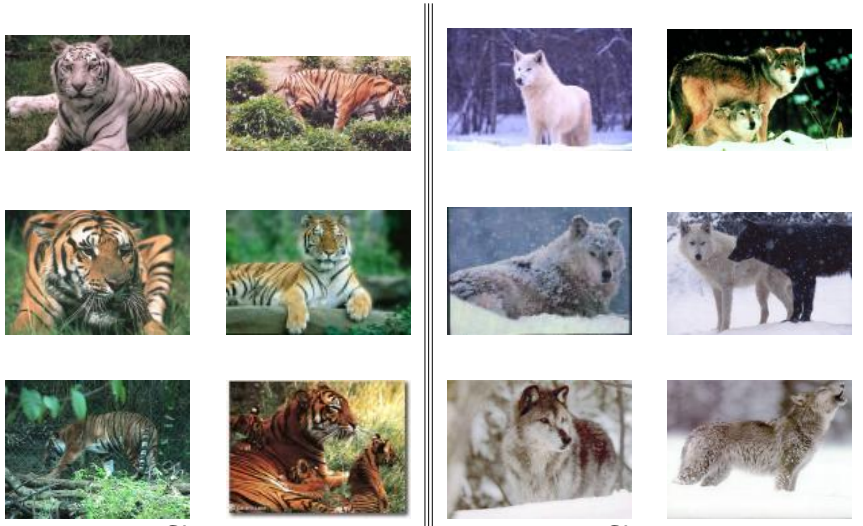
- Complex machines can **over-fit**

  over-fitting: *fitting the training data well at the cost of getting poorer generalisation performance*

- Three red cars. . .

- If we used an infinitely flexible machine we can fit our training data perfectly, but would have no generalisation ability

Binary Classification Task for You



Class 1 Class 2

Which Category?

- Which category does the following image belong to?



Spurious Rules

- You ask a learning machine to solve a task based on data
- It will find a rule that does this, but not necessarily the rule you had in mind—machine learning can't read your mind
- Infinitely flexible machines have an infinity of spurious rules they can learn—they are useless
- What should we do?

All Binary Functions

$$x_0 = 000 \quad y_0 = \begin{cases} 0 \\ 1 \end{cases}$$

$$x_1 = 100 \quad y_1 = \begin{cases} 0 \\ 1 \end{cases} \quad \text{unseen}$$

$$x_2 = 010 \quad y_2 = \begin{cases} 0 \\ 1 \end{cases}$$

$$x_3 = 110 \quad y_3 = \begin{cases} 0 \\ 1 \end{cases}$$

$$x_4 = 001 \quad y_4 = \begin{cases} 0 \\ 1 \end{cases} \quad \text{seen}$$

$$x_5 = 101 \quad y_5 = \begin{cases} 0 \\ 1 \end{cases}$$

$$x_6 = 011 \quad y_6 = \begin{cases} 0 \\ 1 \end{cases}$$

$$x_7 = 111 \quad y_7 = \begin{cases} 0 \\ 1 \end{cases} \quad \text{unseen}$$

$$\mathcal{D} = \{(000, 0), (010, 1), (110, 1), (001, 0), (101, 0)\}$$

Are MLPs Universal Approximators?

- Yes and No
- Yes: If you give me any (continuous) function, I can find an MLP that approximates that function to any desired accuracy (on a bounded region)
- No: If you give me an MLP, I can find a function with an arbitrary high approximation error
- Would an MLP that could approximate any function be useful?
- **Absolutely not!**

Outline

1. Over-fitting?
2. **Controlling Complexity**
3. Hidden structure
4. Regularisation
5. Double Descent



Controlling Complexity

- Infinitely flexible machines don't generalise (because any unseen data could have any value)
- **Machine learning only works because there is some structure in the data**
- A successful machine should capture this structure
- Even deep learning machines with millions or billions of parameters only work because they successfully capture the structure of images or text
- Different learning machines have different performance on different problems because the data has different structure

Training Examples

- As we increase the number of training examples, we make it harder to find a spurious rule
- Bigger data sets allow us to use more complicated machines
- Part of the success of deep learning is because it uses huge training sets—but this is only a part of their success
- (Labelled) data is often expensive to collect so we sometimes have no choice but to use a small training set
- One of the limitations of using deep learning comes because we often have limited data

Preprocessing

- Structure might often be obscure to the learning machine
- If we are trying to predict the spread of disease then a list of place names might be a lot less useful than their coordinates
- Imposing an ordering on an unordered set might **not** be useful

$$\{\text{“blue”} : 0,\ \text{“brown”} : 1,\ \text{“green”} : 2,\ \text{“black”} : 3\}$$

- Choosing an encoding that reflect meaningful structure is essential to successful machine learning

Outline

1. Over-fitting?
2. Controlling Complexity
3. **Hidden structure**
4. Regularisation
5. Double Descent



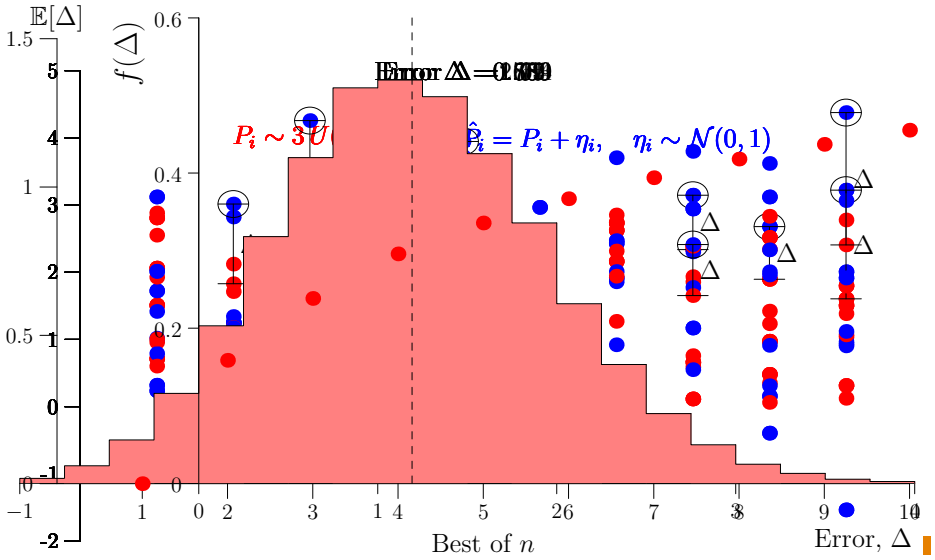
Hidden Structure

- Often the structure of data is invisible to us
- A very successful strategy is to try many different machine learning techniques and choose the best (stupid but effective)
- Often learning machines have adjustable parameters (hyper-parameters) that we have to set (they are the same for all input data, but change with the problem)
- We need to choose the hyper-parameters to fit the data in our problem
- Fine tuning hyper-parameters is important and almost always required (true in SVMs, MLP, deep learning)

Measuring Generalisation Performance

- Recall, we want to predict **unseen** data
- **You cannot use data that you have trained on!**—you will overfit
- Need to split your data up into training and validation set
- Use the validation set to choose the hyper-parameters
- You need a separate testing set if you want to measure your generalisation performance

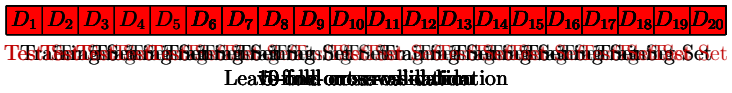
The Overfitting Game



Cross Validation

- If you want to use more data for training then you can use cross validation
- K-fold cross validation splits the data into K groups

$$\mathcal{D} = \{D_i\}_{i=1}^{P} \quad D_i = (\mathbf{x}_i, y_i)$$

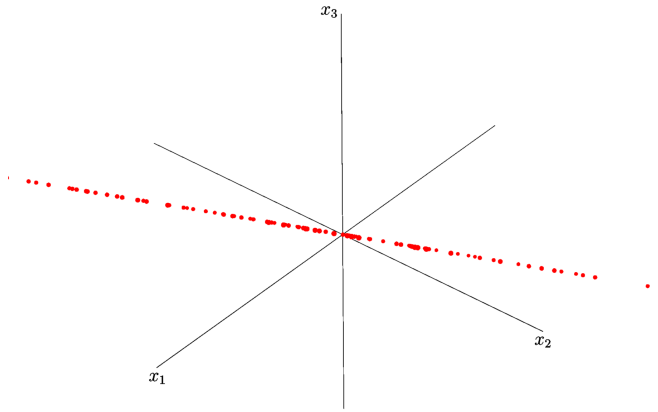


$$\langle E_{gv} \rangle = \frac{5.5 + 8.5 + 1.8 + 3.8 + 3.7 + 3.6 + 7.4 + 4.6 + 0.99 + 4.5 + 4.6 + 5.4 + 6.2 + 3.3 + 2.7}{15} = 4.3$$

- Leave-one-out cross-validation is extreme case

Hidden Structure

Can't spot low dimensional data by looking at numbers



Feature Selection

- Spurious features will allow us to find spurious rules (**over-fitting**)
- We can try different combinations of features to find the best set, although it rapidly becomes intractable to do this in all ways
- We can use heuristics to decide which features to keep (none is fail-safe), or project onto a few principal components (PCA)
- Feature selection however can be powerful, often we can get very good results by keeping only a few variables
- As well as possibly improving generalisation we also get a more **interpretable** rule

Normalising Features

- Measuring a feature in millimeters or kilometers is going to make a lot of difference to the size of that feature
- Many learning algorithms are sensitive to the size of a feature (larger features are more important)
- If we don't know how important different features are then it makes sense to normalise our features, E.g.

$$x_i^\alpha \leftarrow \frac{x_i^\alpha - \hat{\mu}_i}{\hat{\sigma}_i}, \quad \hat{\mu}_i = \frac{1}{m}\sum_{\beta=1}^{m} x_i^\beta, \quad \hat{\sigma}_i^2 = \frac{1}{m-1}\sum_{\beta=1}^{m}(x_i^\beta - \hat{\mu}_i)^2$$

Outline

1. Over-fitting?
2. Controlling Complexity
3. Hidden structure
4. **Regularisation**
5. Double Descent



Explicit Regularisation

- As in ridge regression, we can modify our error function to choose smoother functions

$$L = \sum_{k=1}^{m} \left(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}_k - y_k\right)^2 + \eta\|\boldsymbol{w}\|^2$$

  (Good to normalise features)

- Second term is minimised when $w_i = 0$
- If w_i is large then

$$f(\boldsymbol{x}|\boldsymbol{w}) = \boldsymbol{w}^\mathsf{T}\boldsymbol{x} = \sum_{i=1}^{p} w_i x_i$$

  varies rapidly as we change x_i

Regularisers Are Priors

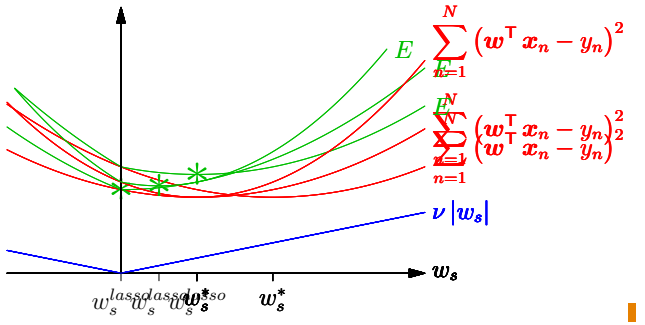
- Recall MAP: maximise $\log f(\mathcal{D}|\boldsymbol{w}) + \log f(\boldsymbol{w})$, so $-\log f(\boldsymbol{w})$ is a regulariser
- Gaussian noise of variance σ^2 and a Gaussian prior $w_i \sim \mathcal{N}(0, s^2)$ give ridge regression with $\eta = \sigma^2/s^2$
- A Laplace prior $f(w_i) \propto \mathrm{e}^{-|w_i|/b}$ gives the **lasso** penalty $\eta\sum_i |w_i|$
- The prior says what we believe before seeing data: small weights, or few non-zero weights
- Regularisation is how we tell the machine about the structure we expect

Lasso

- We can use other regularisers

$$L = \sum_{k=1}^{m} \left(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}_k - y_k\right)^2 + \eta\sum_{i=1}^{p} |w_i|$$

- Spurious features (e.g. colour of flag on energy consumption) will give us a small improvement in training error



Early Stopping Is Implicit Ridge

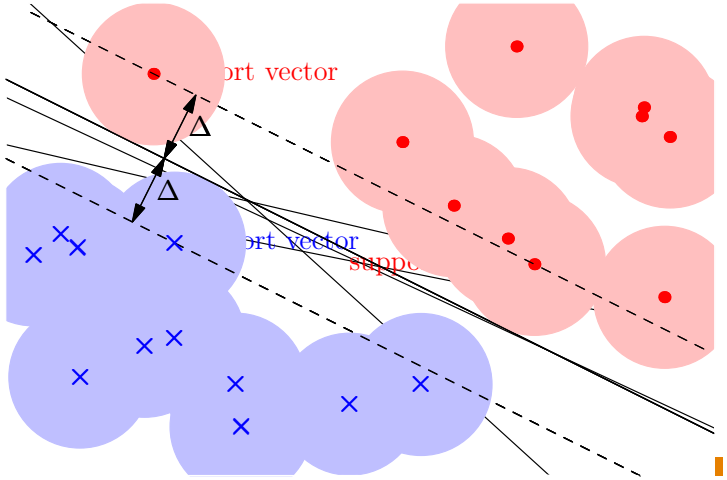
- Some machines regularise without an explicit penalty
- Gradient descent on $\frac{1}{2}\|\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}\|^2$ from $\boldsymbol{w} = \boldsymbol{0}$ with learning rate r: in the SVD basis, after t steps component k is

$$\left(1 - (1 - r s_k^2)^t\right) \times \text{(least squares component)}$$

- Ridge regression multiplies the same component by $s_k^2/(s_k^2 + \eta)$
- Both keep the directions with large s_k and suppress those with small s_k, where the variance lives
- So stopping early acts like ridge with $\eta \approx 1/(rt)$: training time is a regulariser

Maximum Margin Machines

- Perceptrons have many options to separate data



- SVMs choose the machine with the biggest margins

Why SVMs Generalise

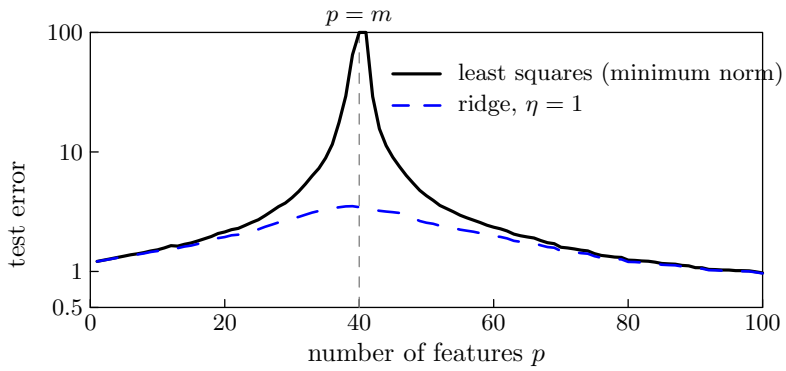
- For data in a ball of radius R separated with margin γ, the capacity of the machine depends on $(R/\gamma)^2$, not on the dimension of the feature space
- That is why the kernel trick, with enormous (even infinite) feature spaces, need not over-fit
- Removing a training point that is not a support vector leaves the solution unchanged, so the leave-one-out error is at most (number of support vectors)$/m$
- Few support vectors and a large margin mean good generalisation
- But you still need to normalise the features and tune C (and the kernel parameters)

Outline

1. Over-fitting?
2. Controlling Complexity
3. Hidden structure
4. Regularisation
5. **Double Descent**



Double Descent



- Least squares on $m = 40$ points using p of 100 features
- The classical U-curve, a spike at $p = m$, then a second descent

Why Bigger Can Be Better

- At $p = m$ there is exactly one fit through every (noisy) point; the smallest singular value is near 0, so the variance $\sum_k \sigma^2/s_k^2$ explodes
- For $p > m$ there are infinitely many exact fits; we take the one with the smallest $\|\boldsymbol{w}\|$
- Gradient descent from $\boldsymbol{w} = \mathbf{0}$ finds exactly this minimum-norm solution: an implicit regulariser
- Modern networks have far more parameters than data, yet trained with (S)GD they often generalise well
- Counting parameters is not the same as measuring complexity

Lessons

- Machine learning isn't magic
- It works when the learning machine is well attuned to the problem
- Sometimes you can help by preprocessing your data
- Regularisers (explicit, priors, or implicit like early stopping and minimum norm) select simpler machines
- Most machines have hyper-parameters that you tune to match the machine to the data
- Complexity is not just the number of parameters: margins and norms matter